

ITPROFEL2: Open Source Programming

Learning Module: Finals - Week 5

Instructor: Henry V. Dela Cruz | Topic: Class Attributes and Methods

I. Overview and Objectives

Last week, we mapped out empty class blueprints. This week, we breathe life into them by defining data that is shared across the entire blueprint, and creating functions that belong exclusively to the objects we create.

Intended Learning Outcomes (ILOs):

- Instantiate a class to create and access class objects.
- Define and modify class attributes dynamically during runtime.
- Create methods inside a class to define object behavior.
- Analyze the structural difference between class-level data and procedural logic.

II. Instantiation and Access

Instantiation is the process of generating a real, usable object from a class blueprint in memory. Once an object is instantiated, you can access its contents using "dot notation" (e.g., `object.attribute` or `object.method()`).

```
class Server:
    pass

node1 = Server() # We have just instantiated node1
node2 = Server() # node2 is a completely separate object in memory
```

III. Defining Class Attributes

A **class attribute** is a variable defined directly inside the class body. This data is shared by *all* instances of the class. It is highly memory efficient for storing constants or default configurations.

```
class Server:
    # Class attribute: Every Server object shares this exact string
    domain = "plsp.edu.ph"

node1 = Server()
print(node1.domain) # Output: plsp.edu.ph

# Modifying a class attribute dynamically changes it for ALL objects
Server.domain = "open-source.org"
print(node1.domain) # Output: open-source.org
```

IV. Creating Methods Inside a Class

Methods are functions that exist inside a class. They define what an object can *do*. Every method must have at least one parameter, conventionally named `self`, which allows the object to refer to itself.

```
class Server:
    domain = "plsp.edu.ph"

    # Creating a method. Notice the 'self' parameter!
    def ping(self):
        print("Ping successful. Server is reachable.")

node1 = Server()
node1.ping() # We call the method using dot notation
```

Laboratory Session: Entity Behavior Injection

Scenario: You are taking the empty `UserEntity` blueprint from last week and injecting shared rules and basic behaviors. This object needs to establish a fixed origin server, and it needs a mechanism to verify its own network connection.

Instructions:

1. Define the class `UserEntity`.
2. Create a **class attribute** named `institution` and assign it the string "PLSP Network".
3. Define a method called `verify_connection(self)`. When called, it should print exactly: "[OK] Entity connected to server."
4. Outside the class, instantiate an object named `system_user = UserEntity()`.
5. Print the `institution` attribute of your object.
6. Call the `verify_connection()` method on your object.
7. Dynamically modify the class attribute `UserEntity.institution` to "Open Source Global".
8. Print the `institution` attribute of your object again to prove the modification affected the instance.

Expected Terminal Output:

```
Current Institution: PLSP Network
[OK] Entity connected to server.
--- Performing Dynamic Update ---
New Institution: Open Source Global
```

Submit your `.py` file. Your entity is evolving; keep the code safe!